

Unified Explainable AI Interface: Multi-modal (Audio + Image) Classification with Comparative Explainable AI

TD Group: DIA 5

Team: Hugo ROBIN, Ghadi SALAMEH, Hector MELL MARIOLLE, Jules SAYAD-BARTH

2026-01-05

Abstract

This project implements a unified Streamlit application for inference and explainable AI (XAI) across two modalities: (i) audio deepfake detection from `.wav` files and (ii) chest X-Ray multi-label pathology scoring from `.png/.jpg/.jpeg` images. The X-Ray pipeline uses two pre-trained models from Hugging Face: `shreydan/CheXpert-5-convnextv2-tiny-384` (a ConvNeXtV2 model fine-tuned on CheXpert with AUROC scores up to 0.93) and `codewithdark/vit-chest-xray` (a Vision Transformer achieving 98.46% validation accuracy). The system exposes LIME, SHAP, and Grad-CAM explanations, and introduces a ViT-specific attention rollout heatmap when Grad-CAM is not applicable. A central design objective is to satisfy the TD requirements through a single workflow that automatically detects the input modality, filters incompatible explanation methods, and provides a side-by-side comparison view. The audio pipeline converts uploaded waveforms into MEL spectrogram images (224×224) and applies a TensorFlow/Keras SavedModel stored locally. The image pipeline loads PyTorch models through the Transformers library from local model directories and computes multi-label probabilities via a sigmoid transformation over logits. The application implements a comparison view that renders multiple XAI outputs for the same input and caches results in the Streamlit session state to avoid expensive recomputation. All implementation claims in this report are backed by the repository artifacts referenced as `path::function`.

1 Introduction

The project addresses two complementary classification settings. First, deepfake audio detection aims to distinguish authentic speech from synthetic or manipulated audio; such models often operate as black boxes, motivating explainability methods to improve transparency and trust during a demonstration. Second, chest X-Ray analysis is implemented as a multi-label pathology scoring task (not a binary “cancer” classifier), where model outputs represent probabilities for a fixed set of thoracic findings.

The TD specification requires multi-modal inputs, mandatory XAI methods (LIME, SHAP, Grad-CAM), automatic filtering of incompatible choices, and a comparison/visualization area. This repository consolidates these requirements into a single Streamlit interface with a unified upload step and a consistent “predict then explain” flow.

2 Repository Analysis (Evidence-Based)

2.1 User Interface Framework

The graphical user interface is implemented with Streamlit (see: `Streamlit/app.py::main`), and dependencies pin `streamlit==1.10.0` (see: `Streamlit/requirements.txt`). The application provides multiple pages in the sidebar, including a unified page named “Unified” (see: `Streamlit/app.py::main`).

2.2 Audio Pipeline

The audio pipeline accepts `.wav` uploads and writes bytes to disk under `audio_files/` (see: `Streamlit/app.py::save_audio_bytes`). Feature extraction is performed by computing a MEL spectrogram with Librosa and rendering it deterministically as a 224×224 RGB image using a fixed Matplotlib figure size and DPI (see: `Streamlit/app.py::create_spectrogram`). The classifier is loaded from a local TensorFlow SavedModel directory `Streamlit/saved_model/model/` (see: `Streamlit/app.py::homepage_audio` and `Streamlit/app.py::unified_page`). Inference is executed on a single example batch by normalizing the spectrogram image to $[0, 1]$ and calling `model.predict` (see: `Streamlit/app.py::predictions_audio`). Output classes are `["real", "fake"]` (see: `Streamlit/app.py::audio_class_names`).

2.3 Image (X-Ray) Pipeline

The X-Ray pipeline loads one of two offline Hugging Face snapshots stored under `Streamlit/saved_model/`: the directories `chexpert_convnextv2_tiny_384` and `vit_chest_xray`. Path constants are defined in `Streamlit/app.py` as `XRAY_CONVNEXT_DIR` and `XRAY_VIT_DIR`. Both models are loaded with `AutoImageProcessor` and `AutoModelForImageClassification` from the Transformers library, using `local_files_only=True` and `torch.float32`, then moved to CPU. The loading logic is in function `load_xray_model`. Preprocessing converts inputs to RGB and applies the HF processor; the code also creates a grayscale preview for UI display, as implemented in `preprocess_xray`. Inference returns logits which are transformed into multi-label probabilities using `torch.sigmoid`, as implemented in `predictions_xray`. Labels are enforced explicitly by writing `id2label` and `label2id` into the model configuration. The label sets are defined in constants `CONVNEXT_CHEXPRT5_LABELS` and `VIT_CHEST_XRAY_LABELS`.

2.3.1 Pre-trained Model References

The two X-Ray models used in this project are publicly available on Hugging Face and are cited below with their original training details.

Model 1: CheXpert-5 ConvNeXtV2 Tiny (384). This model is published under the identifier `shreydan/CheXpert-5-convnextv2-tiny-384`¹. It is a fine-tuned version of `facebook/convnextv2-tiny-22k-384` on the CheXpert dataset. The model outputs multi-label probabilities for five thoracic findings: Atelectasis, Cardiomegaly, Consolidation, Edema, and Pleural Effusion. According to the model card, evaluation AUROC scores are: Atelectasis 0.7943, Cardiomegaly 0.8187, Consolidation 0.9269, Edema 0.9233, and Pleural Effusion 0.9315. The model contains approximately 27.9 million parameters and is distributed under the Apache-2.0

¹<https://huggingface.co/shreydan/CheXpert-5-convnextv2-tiny-384>

license. Training hyperparameters included a learning rate of 0.001, batch size of 32, six epochs, and a cosine learning rate scheduler with 2500 warmup steps.

Model 2: ViT Chest X-Ray. This model is published under the identifier `codewithdark/vit-chest-xray`². It is a fine-tuned Vision Transformer (ViT) based on `google/vit-base-patch16-224-in21k`, also trained on the CheXpert dataset. The model outputs multi-label probabilities for five classes: Cardiomegaly, Edema, Consolidation, Pneumonia, and No Finding. According to the model card, the final validation accuracy is 98.46% with a validation loss of 0.0980. The model contains approximately 85.8 million parameters and is distributed under the MIT license. Training hyperparameters included the AdamW optimizer with a learning rate of 3×10^{-5} , batch size of 32, ten epochs, and mixed precision training.

Table 1 summarizes the key characteristics of both X-Ray models.

Table 1: Summary of X-Ray models used in this project.

Characteristic	ConvNeXtV2 (shreydan)	ViT (codewithdark)
Hugging Face ID	shreydan/CheXpert-5-convnextv2-tiny-384	codewithdark/vit-chest-xray
Base Model	facebook/convnextv2-tiny-22k-384	google/vit-base-patch16-224-in21k
Architecture	ConvNeXtV2 Tiny	Vision Transformer (ViT-Base)
Parameters	27.9M	85.8M
Input Size	384×384	224×224
Output Labels	Atelectasis, Cardiomegaly, Consolidation, Edema, Pleural Effusion	Cardiomegaly, Edema, Consolidation, Pneumonia, No Finding
Best Metric	AUROC up to 0.9315 (Pleural Effusion)	Validation Accuracy 98.46%
Training Dataset	CheXpert	CheXpert
License	Apache-2.0	MIT

2.4 Explainability Implementations and Invocation

LIME is implemented for audio spectrogram images and X-Ray images using the `LimeImageExplainer` class from the `lime.lime_image` module. The audio implementation is in `lime_predict_audio` and the X-Ray implementation is in `lime_predict_xray`, both located in `Streamlit/app.py`. Both paths define a wrapper `predict_fn` to enforce stable dtype and batch shape before calling the underlying model.

SHAP explanations are implemented using a model-agnostic image masker from `shap.maskers.Image` with a blur baseline and `shap.Explainer`. The audio implementation is in `shap_predict_audio` and the X-Ray implementation is in `shap_predict_xray`, both in `Streamlit/app.py`. The X-Ray SHAP wrapper passes images through the HF processor and returns multi-label probabilities via sigmoid.

Grad-CAM is implemented in two contexts. For audio, Grad-CAM is computed on the actual TensorFlow/Keras classifier by locating the last Conv2D layer and backpropagating gradients with `tf.GradientTape`; see function `grad_predict_audio` in `Streamlit/app.py`. For X-Ray, Grad-CAM is implemented for convolutional HF models via PyTorch forward/backward hooks over the last Conv2d module encountered; see function `grad_cam_hf_image_model` in `Streamlit/app.py`.

²<https://huggingface.co/codewithdark/vit-chest-xray>

For ViT models, the repository provides an attention rollout heatmap based on transformer attentions, as a practical explanation alternative where Grad-CAM is not applicable; see function `vit_attention_rollout_xray` in `Streamlit/app.py`.

2.5 Compatibility Filtering (Automatic)

Automatic XAI compatibility filtering is implemented in the unified UI. The modality is detected by checking the uploaded filename extension; the method selector is built from modality- and model-specific lists (see: `Streamlit/app.py::unified_page`). For X-Ray, when the ViT model is selected, the UI offers LIME, SHAP, and Attention Rollout, while Grad-CAM is omitted; for ConvNeXtV2, the UI offers LIME, SHAP, and Grad-CAM (see: `Streamlit/app.py::unified_page`).

2.6 Comparison / Side-by-Side Visualization

The comparison view is implemented in the unified page as a “Compare” mode. Multiple XAI outputs are rendered side-by-side using Streamlit columns, and the predicted label or top pathology is displayed alongside the explanations. The implementation is in `unified_page` within `Streamlit/app.py`, specifically in the helper functions `_show_audio_compare` and `_show_xray_compare`. The repository also provides a Streamlit 1.10 compatibility fallback because `st.tabs` may be unavailable; in that case a `st.radio` view selector is used.

2.7 Model Loading, Checkpoints, and Environment

Audio weights are stored as a TensorFlow SavedModel in the directory `Streamlit/saved_model/model/`. X-Ray weights are stored as offline HF snapshots containing `config.json`, `model.safetensors`, and `preprocessor_config.json`, located under `Streamlit/saved_model/`. The environment is constrained by TensorFlow 2.6.2 and NumPy 1.19.5; PyTorch is pinned to 1.13.1 to maintain compatibility with the required typing extensions. These constraints are specified in `Streamlit/requirements.txt`. The code selects CPU execution for X-Ray models in function `load_xray_model`; the audio TensorFlow model uses the default device behavior.

2.8 Original Repositories and Integration (Evidence)

The README references assets from an external deepfake audio XAI repository via image URLs. The About page (function `about`) accurately describes the current runtime: a single TensorFlow/Keras CNN classifier for audio deepfake detection (based on the methodology from the Deepfake-Audio-Detection-with-XAI repository), and two Hugging Face models for X-Ray pathology detection (ConvNeXtV2 and ViT). The original audio repository explored multiple architectures (VGG16, MobileNet, ResNet, Custom CNN); this implementation uses a single trained model. The X-Ray models are offline snapshots from Hugging Face with explicit model citations in the About page.

3 Requirements and Objectives Traceability

Requirement	Implementation	Evidence (file/function)	Status
Multi-modal up-load (.wav + X-Ray image)	Single uploader with extension-based routing	Streamlit/app.py::unified_page	OK
Model selection	X-Ray: ConvNeXtV2 vs ViT; Audio: single model (no selector)	Streamlit/app.py::unified_page, Streamlit/app.py::load_xray_model	Partial
LIME support	Audio and image LIME explainers	Streamlit/app.py::lime_predict_audio, Streamlit/app.py::lime_predict_xray	OK
SHAP support	Audio and image SHAP explainers with image masker	Streamlit/app.py::shap_predict_audio, Streamlit/app.py::shap_predict_xray	OK
Grad-CAM support	Audio Grad-CAM (Keras Conv2D), X-Ray Grad-CAM for conv HF models	Streamlit/app.py::grad_predict_audio, Streamlit/app.py::grad_cam_hf_image_model	OK
Automatic compatibility filtering	XAI list depends on modality and X-Ray architecture	Streamlit/app.py::unified_page	OK
Basic workflow (input→preprocess→predict→postprocess)	Implemented for both modalities in unified	Streamlit/app.py::create_spectrogram, predictions_audio, preprocess_xray, predictions_xray, unified_page	OK
Comparison/visualization tab	Side-by-side columns in Compare view; includes prediction context	Streamlit/app.py::unified_page (_show_*_compare)	OK
Documentation and setup	README + pinned requirements + run instructions	README.md, Streamlit/requirements.txt	OK

4 System Architecture

Figure 1 summarizes the core components and their connections. The system is implemented as a single Streamlit module, with the unified page acting as the controller that selects pipelines and explanation strategies.

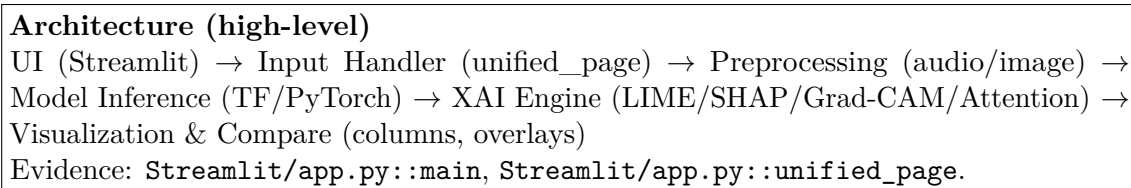


Figure 1: High-level system architecture (text diagram).

5 Implementation Details

5.1 Unified Interface (UI/UX Flow)

The unified interface is implemented in the function `unified_page` within `Streamlit/app.py`. It provides a single uploader, performs modality detection by checking the filename suffix, then invokes the corresponding preprocessing and inference pipeline. The page renders three views (Prediction, Explain, Compare) using `st.tabs` when available and a radio fallback for Streamlit 1.10 compatibility. The user selects explanation methods via a multiselect whose options are automatically filtered based on modality and X-Ray model selection.

5.2 Audio Deepfake Detection Pipeline

Uploaded audio is stored to disk and transformed into a MEL spectrogram image using Librosa. The spectrogram rendering is deterministic and fixed to 224×224 pixels to preserve consistency with the model’s training pipeline; see function `create_spectrogram`. The classifier is a TensorFlow/Keras SavedModel loaded from `Streamlit/saved_model/model/` and used in batch size 1 inference; see function `predictions_audio`. The audio detection approach is based on the methodology from the Deepfake-Audio-Detection-with-XAI repository, which explores multiple architectures (VGG16, MobileNet, ResNet, Custom CNN) for spectrogram-based classification.

5.3 Lung Pathology X-Ray Pipeline

The X-Ray pipeline uses Transformers to load local model snapshots obtained from Hugging Face. Input images are converted to RGB and processed via the HF image processor; see function `preprocess_xray`. Inference outputs are multi-label probabilities computed by applying a sigmoid to logits; see function `predictions_xray`. Labels are explicitly enforced to match the project-defined pathology sets in function `load_xray_model`.

The first model, `shreydan/CheXpert-5-convnextv2-tiny-384`, is a ConvNeXtV2 architecture fine-tuned on CheXpert with five output labels: Atelectasis, Cardiomegaly, Consolidation, Edema, and Pleural Effusion. The second model, `codewithdark/vit-chest-xray`, is a Vision Transformer fine-tuned on CheXpert with five output labels: Cardiomegaly, Edema, Consolidation, Pneumonia, and No Finding. Both models were trained on the CheXpert dataset and achieve strong performance on their respective evaluation metrics (see Section 2.3.1 for detailed metrics). The repository stores offline snapshots of these models under `Streamlit/saved_model/`.

5.4 Explainability Methods (XAI)

LIME operates on image-like inputs in both modalities. The audio spectrogram is treated as an image and explained via superpixel perturbations; see function `lime_predict_audio`. A methodological limitation is that LIME superpixels on spectrograms do not map cleanly to time-frequency structure; results should be interpreted as approximate regions of interest rather than precise frequency bands. X-Ray LIME perturbs the input image and routes perturbed samples through the HF processor and model; see function `lime_predict_xray`.

SHAP uses an image masker with a blur baseline and a model-agnostic explainer for both modalities; see functions `shap_predict_audio` and `shap_predict_xray`. Evaluation budgets are specified via the `max_evals` parameter.

Grad-CAM is implemented for the actual audio classifier model using func-

tion `grad_predict_audio` and for convolutional HF image models using function `grad_cam_hf_image_model`. For ViT models, the code provides attention rollout via function `vit_attention_rollout_xray` and the UI excludes Grad-CAM accordingly.

5.5 Compatibility Filtering Logic

Compatibility is expressed as UI-level option selection rather than a separate registry. In the unified page, the XAI options are set to LIME, SHAP, and Grad-CAM for audio; LIME, SHAP, and Grad-CAM for ConvNeXtV2; and LIME, SHAP, and Attention Rollout for ViT. This prevents invalid selections without requiring runtime error handling; see function `unified_page`.

5.6 Comparison / Visualization

The comparison view renders multiple explanations side-by-side in columns and includes the classification result as context text. The implementation caches computed explainers and heatmaps in `st.session_state` to reduce runtime cost when the same input is explored repeatedly; see function `_xai_cached` and its usage in `unified_page`. For SHAP in comparison, the visualization uses Matplotlib overlays because raw SHAP arrays include negative values that are not directly displayable via `st.image` in Streamlit 1.10.

6 Refactoring and Integration Decisions

The repository consolidates multi-modal inference and XAI into a single Streamlit file. Evidence of integration from earlier codebases appears in the README assets and in the legacy text of the About page. In the current code, the engineering trade-off favors TD deliverability (single entry point, local checkpoints, deterministic preprocessing, UI compatibility workarounds) over modularity and automated testing. The code also pins dependency versions to reconcile TensorFlow and PyTorch compatibility constraints (see: `Streamlit/requirements.txt`).

7 Testing and Validation

No automated tests are found in the repository (no `tests/` directory and no test runner configuration). A minimal validation plan for a TD demonstration should include: modality routing checks (audio vs image), sanity checks on output shapes and probability ranges, successful generation of each XAI method for at least one example, verification that incompatible methods are not offered in the UI, and confirmation that the comparison view displays at least two methods side-by-side for a single input.

8 Results and Demonstration Workflow

The repository contains example audio files in the project root: `cascade_real_5s.wav` and `cascade_fake_5s.wav`, which can be used for demonstration. A recommended demo flow is as follows: launch the app, select the unified page, upload an audio file to show prediction and at least two explanations (e.g., LIME and Grad-CAM), then upload an X-Ray image, select the ConvNeXtV2 model to show Grad-CAM plus LIME and SHAP, and finally switch to the Compare view to show side-by-side explanations with the prediction context text. All steps correspond to existing UI behavior implemented in functions `main` and `unified_page`.

9 Limitations and Future Work

The application uses computationally expensive explainers, particularly SHAP, and mitigates cost only through session caching rather than offline precomputation. The Grad-CAM implementation for HF models selects the last convolutional layer encountered, which is a reasonable heuristic but may not always yield the most interpretable maps across architectures. The X-Ray task implemented in this repository is multi-label thoracic pathology scoring with five labels for each model snapshot; a strict “lung cancer detection” classifier is not found in the current runtime logic. Future work should consider modularization, automated tests, exportable results (images/reports), and explicit calibration or thresholding for pathology decision support.

10 Setup and Reproducibility

Dependencies and version constraints are specified in `Streamlit/requirements.txt`. The README provides run commands: `pip install -r Streamlit/requirements.txt` followed by `cd Streamlit` and `streamlit run app.py` (see: `README.md`). Local model directories required at runtime are stored under `Streamlit/saved_model/` (see repository tree).

Generative AI Usage Statement

We used Cursor AI with Claude Opus 4.5 to assist with code corrections and improvements after an initial implementation of the solution. We also used ChatGPT to help understand the project requirements and to provide feedback on our code and report for potential mistakes we might have missed during review. We transparently disclose the use of these generative AI tools and remain fully responsible for all technical decisions, validation, and submitted content.

References

1. **Deepfake-Audio-Detection-with-XAI.** Guri10/Deepfake-Audio-Detection-with-XAI. GitHub Repository. Available at: <https://github.com/Guri10/Deepfake-Audio-Detection-with-XAI>. Original audio deepfake detection project exploring VGG16, MobileNet, ResNet, and Custom CNN architectures with LIME, Grad-CAM, and SHAP explanations. Authors: Aamir Hullur, Atharva Gurav, Aditi Govindu, Parth Godse.
2. **shreydan/CheXpert-5-convnextv2-tiny-384.** Hugging Face Model Hub. Available at: <https://huggingface.co/shreydan/CheXpert-5-convnextv2-tiny-384>. Fine-tuned ConvNeXtV2 Tiny model on CheXpert dataset for multi-label thoracic pathology classification (5 labels). Base model: facebook/convnextv2-tiny-22k-384. License: Apache-2.0.
3. **codewithdark/vit-chest-xray.** Hugging Face Model Hub. Available at: <https://huggingface.co/codewithdark/vit-chest-xray>. Fine-tuned Vision Transformer (ViT) on CheXpert dataset for chest X-ray classification (5 labels). Base model: google/vit-base-patch16-224-in21k. License: MIT.
4. **CheXpert Dataset.** Irvin, J., et al. (2019). CheXpert: A Large Chest Radiograph Dataset with Uncertainty Labels and Expert Comparison. Proceedings of the AAAI Conference on Artificial Intelligence.

5. **LIME**. Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). “Why Should I Trust You?”: Explaining the Predictions of Any Classifier. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.
6. **SHAP**. Lundberg, S. M., & Lee, S. I. (2017). A Unified Approach to Interpreting Model Predictions. Advances in Neural Information Processing Systems 30.
7. **Grad-CAM**. Selvaraju, R. R., et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. Proceedings of the IEEE International Conference on Computer Vision.